A

Project Report

On

Chat Buddy Application

Submitted in the partial fulfillment for the requirement for the award of

Bachelor of Technology
In

COMPUTER SCIENCE AND ENGINEERING

Guided By:
Mr. Mohinder Singh

Submitted By:
Ayush Kumar(220501010010)
Ishank Rajput(2205010100019)
Mohd.Suhail(2205010100035)
Khushi Chauhan(2205010100026)

RV INSTITUTE OF TECHNOLOGY, BIJNOR

Affiliated to AKTU, Lucknow || 2025-26

Certificate

This is certified that this project entitled "**Chat Buddy Application"** submitted by Mohd. Suhail (2205010100035), Ayush Kumar (2205010100010), Ishank Rajput (2205010100019) and Khushi Chauhan (2205010100026) students of CSE from **R V INSTITUTE OF TECHNOLOGY, BIJNOR** in the partial fulfillment of the requirement for the award of Bachelor of Technology (CSE) Degree from Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a Bona-fide record of students on study carried under my supervision & guidance.

This report has not been submitted to any other university or institution for the award of Any Degree.

Project Guide	**External Examiner**	**Head of Department**
(Mr. Mohinder Singh)	(Mr./Miss…….)	(Mr. Mohinder Singh)

Acknowledgement

With immense pleasure we present the project report as part of the curriculum of the final year in "**Computer Science and Engineering**". We wish to give a special thanks to our H.O.D **Mr. Mohinder Singh** who give us unending support right from the idea was conceived. We also express our sincere and profound thanks to our project guide **Mr. Mohinder Singh** who always stood by us helping and guiding support. We are also thankful to the staff members who always gave us a helping and showed immense faith in us whenever required.

At last, thanks to our director sir who has organized so well and disciplined college and give us a chance to show our potentials and made us the memories of the college by allowing us to complete this project. Thanks to all……

1. Mohd. Suhail
2. Ayush Kumar
3. Khushi Chauhan
4. Ishank Rajput

Abstract

Real-time communication has become an integral part of our digital lives, fostering seamless interaction and collaboration across the globe. In this era of instant connectivity, developing a RealTime Chat Web Application is essential for facilitating swift and efficient communication. This project aims to create such an application using the MERN (MongoDB, Express.js, React.js, Node.js) stack, a powerful combination of technologies for building full-stack web applications.

The proposed Real-Time Chat Web Application will offer users the ability to register, log in securely, and engage in real-time conversations with other users. Key features include real-time messaging, user authentication, user profiles, and the ability to create chat rooms for group discussions.

The backend of the application will be built using Node.js and Express.js, providing a robust and scalable server-side architecture. MongoDB will serve as the database to store user information, chat messages, and other relevant data. The use of WebSocket technology, facilitated by libraries like Socket.io, will enable real-time communication between users, ensuring instant message delivery and updates.

On the frontend, React.js will be utilized to create a dynamic and responsive user interface. Users will be able to view their conversations, send messages, and receive updates in real-time without the need for page refreshes. The application will also be designed with modern UI/UX principles in mind to enhance user experience and engagement.

Overall, this Real-Time Chat Web Application aims to provide a seamless and efficient platform for users to communicate in real-time, leveraging the power and versatility of the MERN stack. Through this project, we seek to contribute to the advancement of real-time communication technologies and empower users with a reliable and user-friendly chat solution.

Declaration

We hereby declare that the project entitled "**Chat Buddy Application**" toward the partial fulfilment for the award of "**BACHELOR OF TECHONOLOGY"** in Computer Science and Engineering submitted in the Department of RVIT, BIJNOR (U.P) is an Authentic record of our work carried out under the kind guidance of Mr. Mohinder Singh.

Name- Mohd. Suhail

Roll. No- 2205010100035

Date- Signature-

Name – Ayush Kumar

Roll. No – 2205010100010

Date- Signature-

Name- Ishank Rajput

Roll. No- 2205010100019

Date- Signature-

Name – Khushi Chauhan

Roll. No – 2205010100026

Date- Signature-

Table of Contents

CONTENTS **PAGE NO.**

Certificate 2
Acknowledgement 3
Abstract 4
Declaration 5

Table of content……..……6

CHAPTER 1……..8
1.0 Introduction……..……....8-12
1.1 Introduction about the project……....8-9
1.2 Aim…….....9
1.3 Features……9-11
1.4 System Requirement……..11-12

CHAPTER 2……....13
2.0 Literature Review.....13-17
2.1 Competitive Analysis.....13 2.2 Communication Protocols.....13-17

CHAPTER 3……....14
3.0 Novelty of the project ……..18-19

CHAPTER 4…….....…20
4.0 Design and Methodology……..…..20-27
4.1 Design of project.....20-25

4.2 Methodology of project.....25-27

CHAPTER 5…….....28
5.0 Experimentation…….……..28-55

5.1 Software Implementation.....28-31
5.2 Coding.....32-55

CHAPTER 6…….....56
6.0 Result and Discussion……..56-63

6.1 Discussion.....56-58

6.2 Result.....58-63

CHAPTER 7…….....64
7.0 Conclusion…….....64-65 **CHAPTER 8…….....66** 8.0 Future Scope……....66-67

CHAPTER 8…….....68 9.0 References ……..……68

CHAPTER 1

Introduction

1.1 Introduction about the project

In today's digitally interconnected world, real-time communication has become an indispensable aspect of our daily lives. With the increasing demand for instant connectivity and seamless interaction, the development of real-time chat applications has gained significant momentum. These applications facilitate instantaneous communication among users, transcending geographical barriers and time constraints.

The MERN stack, comprising MongoDB, Express.js, React.js, and Node.js, stands as a robust foundation for building modern web applications. Leveraging the power of MERN, developers can create dynamic and responsive chat platforms that offer real-time messaging capabilities along with a user-friendly interface.

This project aims to design and implement a real-time chat application using the MERN stack. By harnessing the capabilities of MongoDB for database management, Express.js for server-side logic, React.js for frontend development, and Node.js for server-side scripting, our objective is to deliver a scalable and efficient solution for seamless communication.

In this report, we delve into the detailed architecture, design principles, implementation strategies, and challenges encountered during the development of the real-time chat application. Additionally, we discuss the features, functionalities, and potential extensions of the application, highlighting its significance in the realm of modern communication technologies.

Through this project, we endeavour to showcase the prowess of the MERN stack in crafting innovative and interactive web applications, while also providing insights into the intricacies of realtime communication systems.

The MERN stack offers a comprehensive ecosystem for building scalable and responsive web applications. MongoDB serves as the backbone for storing and managing data, while Express.js facilitates the development of robust server-side applications. React.js empowers frontend development with its component-based architecture and virtual DOM, enabling dynamic and interactive user interfaces. Node.js, known for its event-driven and non-blocking I/O model, powers the server-side logic, ensuring high-performance and real-time capabilities.

1.2 Aim

The aim of this project is to build a functional real-time messaging application for developers by using modern web technologies. Unlike most chat applications available in the market, this one will focus on developers and will attempt to boost their productivity. Although we are not expecting it to have a plethora of utilities due to the limited time frame, sharing code and watching a repository will be our core features. It will be fully open-source. Everyone will be able to dig into the code to read what is going on behind the scenes, or even contribute to the source code. So, it was within our intentions to write clean, scalable code following the most popular patterns and conventions for each of the languages and relevant libraries.

1.3 Features

Here's a list of features you might consider implementing in your Real-Time Chat Web Application using the MERN stack:

-
- **Real-Time Messaging:**

Enable users to send and receive messages instantly without page refresh using WebSocket technology.

- **User Authentication:**

Implement secure user registration and authentication mechanisms to ensure user accounts are protected.

- **User Profiles:**

Allow users to create profiles with avatars, usernames, status updates, and other relevant information.

- **Friend/Follow System:**

Enable users to connect with others by sending friend requests or following their profiles.

- **Private Messaging:**

Allow users to initiate private one-on-one conversations with other users.

- **Message Formatting:**

Support formatting options such as bold, italic, underline, bullet points, etc., to enhance message readability.

- **File Sharing:**

Allow users to share files (images, videos, documents) within chat conversations.

-
- **Message Reactions:**

Allow users to react to messages with emojis or custom reactions to express their feelings or opinions.

- **Message Editing/Deleting:**

Allow users to edit or delete their own messages within a certain time frame to correct errors or remove unwanted content.

- **Search Functionality:**

Enable users to search for specific messages, users, or keywords within chat conversations.

- **Notifications:**

Send real-time notifications to users for new messages, friend requests, mentions, etc., even when they are not actively using the application.

- **Presence Indicators:**

Display online/offline status indicators to show when users are active or inactive.

- **Emojis and Stickers:**

Provide a library of emojis and stickers for users to enhance their messages and express emotions.

1.4 System Requirement

Now, this method is intended in such the way that it takes fewer resources to figure out work

•

correctly. That is the minimum needs that we'd like to require care of:-

The system wants a minimum of two GB of ram to run all the options.

- It wants a minimum 1.3 GHz processor to run smoothly.
- Rest is all up to the user's usage can take care of hardware.
- For security opposing anti-virus is suggested.
- **RAM:** At least 256 MB of RAM. The amount of RAM needed depends on the number of concurrent client connections, and whether the server and multiplexor are deployed on the same host.
- **Disk Space:** Approximately 300 MB required for Instant Messaging Server software.
- **Processor:** Minimum 1.3 gigahertz (GHz) x86- or x64-bit dual core processor with SSE2instruction set and recommended 3.3 gigahertz (GHz) or faster 64-bit dual core processor with SSE2 instruction set.
- **Memory:** Minimum 2-GB RAM and recommended 4-GB RAM or more.

The system is made correctly, and all the testing is done as per the requirements. So, the rest

of the things depend on the user, and no one can harm the data or the software if the proper care is done.

CHAPTER 2

Literature Review

2.1 Competitive Analysis

Competitive Analysis Prior to getting started with the application development, we did some research on the current messaging platforms out there. We were looking forward to building a unique experience, rather than an exact clone of an existing chat platform. We already knew of the existence of several messaging applications, and a few chat applications that suited developers. However, never before had we done an in-depth analysis of their tools to find out whether they were good enough for developers. Soon, we realized that none of the sites were heading in our direction. Some of them were missing features which we considered crucial and others had opportunities for further enhancements. Contrary to what many people think, having a few platforms around is not a necessarily a bad thing. We were able to get ideas of what to build and how and determine which technologies and strategies to use based on their experience. Often, this was as simple as checking their blogs [3]. Companies like Slack regularly post development updates (such as performance reviews, technology comparisons, and scalability posts). Other times, we had to dig into the web to find out the different options we had and pick out the one which we considered to be the most appropriate.

2.2 Communication Protocols

For most web applications, communication protocols are not a subject of discussion. AJAX through HTTP is the way to go since it is reliable and widely supported. However, that is not our case. We need, albeit not in every single situation, an extremely fast communication method to send/receive messages in real time. For messaging, there are a few communication protocols available for the web. The most popular ones are AJAX, WebSocket, and WebRTC. AJAX is a slow approach. Not

only because of the headers that have to be sent in every request, but also, and more important, because there is no way to get notified of new messages in a chat room.

By using AJAX, we would have to request/pull new messages from the server every few seconds, which would result in new messages to take up to a few seconds to appear on the screen, not to say the numerous redundant requests that this would generate. WebSocket are a better approach. WebSocket connections can take up too few seconds to establish, but thanks to the full-duplex communication channel, messages can be exchanged swiftly (averaging few milliseconds delay per message). Also, both client and server can get notified of new requests through the same communication channel, which means that unlike AJAX, the client does not have to send the server a petition to retrieve new messages but rather wait for the server to send them. Let's discuss all communication protocols for web: -

2.2.1 AJAX

Asynchronous JavaScript and XML, while not a technology in itself, is a term coined in 2005 by Jesse James Garrett, that describes a "new" approach to using a number of existing technologies together, including HTML or XHTML, CSS, JavaScript, DOM, XML, XSLT, and most importantly the XML Http Request object. When these technologies are combined in the Ajax model, web applications are able to make quick, incremental updates to the user interface without reloading the entire browser page. This makes the application faster and more responsive to user actions. Although X in Ajax stands for XML, JSON is preferred over XML nowadays because of its many advantages such as being a part of JavaScript, thus being lighter in size. Both JSON and XML are used for packaging information in the Ajax model. The Fetch API provides an interface for fetching resources. It will seem familiar to anyone who has used XMLHTTPRequest, but this API provides a more powerful and flexible feature set. Server-sent events Traditionally, a web page has to send a request to the server to receive new data; that is, the page requests data from the server. With server-sent

events, it's possible for a server to send new data to a web page at any time, by pushing messages to the web page. These incoming messages can be treated as Events + data inside the web page. See also: Using server sent events.

2.2.2 WebSocket

The WebSocket API is an advanced technology that makes it possible to open a two-way interactive communication session between the user's browser and a server. With this API, you can send messages to a server and receive event-driven responses without having to poll the server for a reply. The primary interface for connecting to a WebSocket server and then sending and receiving data on the connection. The event sent by the WebSocket object when a message is received from the server. Either a single protocol string or an array of protocol strings. These strings are used to indicate subprotocols, so that a single server can implement multiple WebSocket sub-protocols (for example, you might want one server to be able to handle different types of interactions depending on the specified protocol). If you don't specify a protocol string, an empty string is assumed. The constructor will throw a Security Error if the destination doesn't allow access. This may happen if you attempt to use an insecure connection (most user agents now require a secure link for all WebSocket connections unless they're on the same device or possibly on the same network). WebSocket is an event-driven API; when messages are received, a message event is sent to the WebSocket object [5]. To handle it, add an event listener for the message event, or use the on-message event handler. To begin listening for incoming data, you can do something like this.

2.2.3 WebRTC

WebRTC or Web Real Time Communications allows the establishment of real time audio and video communications between users' browsers. The creation of the communication link is mediated by a web server with WebRTC capabilities. The link itself can be peer-to-peer between the browsers. WebRTC does not depend on a standardised signalling infrastructure, but rather leverages the web JavaScript environment and standardised browser APIs. This allows for implementations that range from a simple audio communication between two people 17 up to a videoconference with multiple participants, out of the box as part of a normal browser. WebRTC thus has at least three actors involved: a web server and two browsers. The mediation of the usable communication channels is negotiated between these based on a complex set of specifications that are detailed in the WebRTC overview below. Not only is there a complex combination of specifications, but the work is also distributed between the IETF doing the protocol stack and W3C creating the browser APIs [6]. Still WebRTC remains a part of the Web, thus all the vulnerabilities described in the Web Security Guide still apply. But the complexity and the fact that the communication is real time also bring new aspects and vulnerabilities. This case study dives deeper into the vulnerabilities to which user and server assets are exposed by their use of WebRTC. This is not limited to the three actors mentioned, but these may also behave in unexpected ways. Chapter 2 describes the relevant assets, and based on the list of assets, the surface of exposure and the assets targeted and the list of possible malicious actors becomes much clearer. This exposes some of the underlying issues otherwise hidden by the complexity of the combination of WebRTC specifications. The fact that real time communication is involved immediately raises the issue around permissions and timing of access. For communication between two users, the browser needs access to speaker and microphone. This means the browser can potentially be remotely turned into a device to capture all the sounds in a location. It is known that native applications on mobile phones have issues around permissions and how fine grained they ought to be, whereas how coarse they actually are. For WebRTC the issue is the same. This case

study assesses the implications and gives hints on potential security problems in current implementations. A central issue in WebRTC that was discussed widely in the Working Groups but has not yielded satisfactory results is identity management [7]. Am I really sure I am talking to the right person? On the Web, the trusted telephone operator is not always there. The case study looks into user authentication and naming and raises issues around credentials and keying. The case study then goes on with an analysis of a classic cross site scripting attack in a WebRTC scenario. What happens when an attacker can inject arbitrary JavaScript code into the running WebRTC application or even manages to upload a malicious WebRTC application to a server that delivers it to the browser?

The evaluation of exposure is used to describe a landscape of possible attacks.

CHAPTER 3

Novelty of the project

For a Real-Time Chat Web Application using the MERN stack, focusing on novelty involves adding unique features or user experiences that enhance the basic chat functionality.

1.0 Emoji Reactions and Stickers:

- Allow users to react to messages with emojis or stickers, adding a fun and expressive element to conversations.

2.0 Real-Time Chat:

- Implement real-time messaging using technologies like WebSocket or a library like Socket.io to enable instant message delivery. Show real-time typing indicators and read receipts for messages.

3.0 Multimedia Support:

- Enable users to send and receive multimedia files like images, videos, and audio messages. Implement image and video previews within the chat interface.

4.0 Dark Mode:

- Introduce a dark mode theme option for users who prefer a darker interface, enhancing accessibility and user preference.

5.0 Customizable Chat Themes:

- Allow users to customize the appearance of their chat interface with different themes, colours, and fonts, providing a personalized user experience.

6.0 Real-Time Typing Indicators:

- Display real-time typing indicators to show when other users are actively composing messages, enhancing the feeling of live communication.

7.0 User Authentication:

- When a user logs in or registers, authenticate their credentials against your database (MongoDB) using a secure mechanism like bcrypt for password hashing.
- Upon successful authentication, generate a JWT for the user containing their unique identifier (such as user ID) and any necessary claims (e.g., role, expiration time).
- Sign the JWT using a secret key known only to your server to prevent tampering.

By incorporating these novel features into your simple Real-Time Chat Web Application, you can provide users with a more engaging and versatile communication platform while still maintaining simplicity and ease of use.

CHAPTER 4

Design & Methodology

4.1 Design of Project

Designing and implementing a Real-Time Chat Web Application using the MERN stack involves several key components and steps. Here's a high-level overview of the design and methodology:

This study's research design depends on qualitative analysis. Instead of examining many research works, we have subjectively picked key contributions in the field of real-time chat application development. These apps often mix socket.io with frontend languages such as HTML, CSS, and others. It is critical to ensure that the project is adaptable and scalable, as there is a potential need for future expansion. Extensive literature reviews have been conducted to identify various methodologies employed by developers in the development of chat applications.

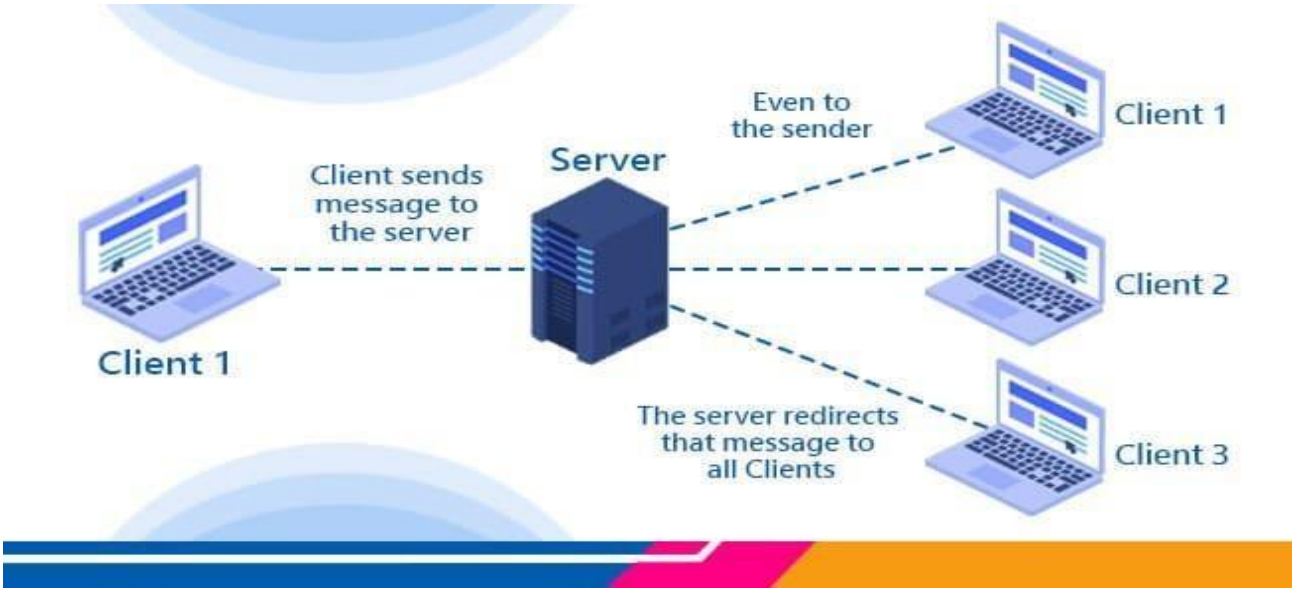

Fig: 4.0 Working Platform

4.1.1 System Architecture:

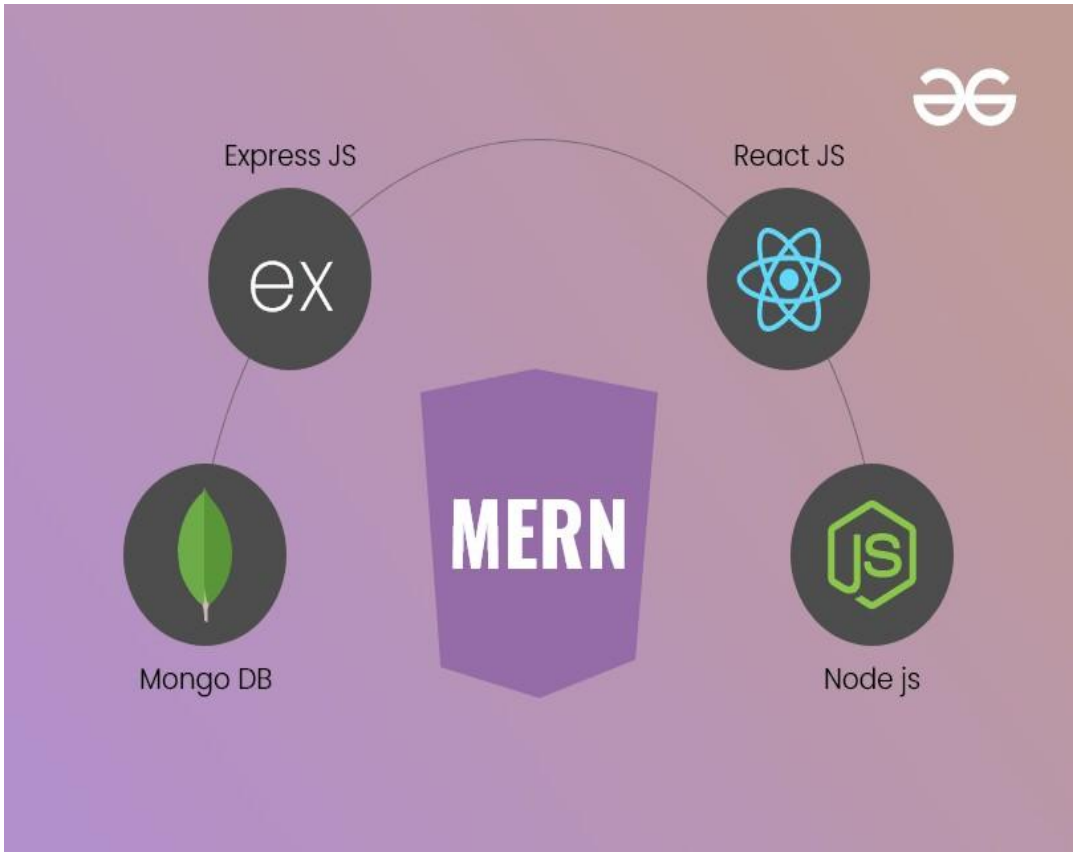

Fig: 4.1 Technology

- ***Frontend (React.js):*** The client-side interface responsible for rendering the chat UI, handling user interactions, and communicating with the backend.
- ***Backend (Node.js, Express.js):*** The server-side application responsible for managing user authentication, storing chat data, and facilitating real-time communication using WebSocket technology.

- ***Database (MongoDB):*** The NoSQL database used to store user data, chat messages, and other application-related information.

The project design will adhere to the MERN stack development approach. This entails utilizing MongoDB as the database, Express.js as the backend framework, React for frontend development, and Node.js as the runtime environment. The MERN stack provides a solid foundation for creating a robust and scalable real-time chat application. For implementation, this project makes use of React.js as the frontend framework. Facebook created React.js, is chosen primarily for its reusable component feature. This feature enables efficient rendering of components without the need to recreate or load them repeatedly. Take Instagram as an example, where the stories feature utilizes reusable components that are rendered for each story. By employing React.js in the frontend, the application can achieve faster performance by reusing components as needed. This approach not only improves speed but also enhances extensibility, allowing for easy addition of new components when required.

4.1.2 User Authentication:

Implement user authentication using JWT (JSON Web Tokens) for secure login and registration. Store user credentials securely in the database, hashing passwords using bcrypt or a similar hashing algorithm. Upon successful authentication, issue a JWT to the client, which will be used to authenticate subsequent requests to the server.

4.1.3Real-Time Communication:

Use WebSocket technology (e.g., Socket.io) to establish a bi-directional, real-time communication channel between the client and server. Implement WebSocket endpoints on the server to handle incoming connections, message transmission, and disconnections.

Handle WebSocket events on the client-side to send and receive messages, update the UI in real-time, and handle user interactions.

Socket.io is a popular JavaScript-based library for online applications. It comprises a server library written in Node.js and a client library that runs in web browsers. Socket.io generally uses the WebSocket protocol from the client's perspective, but it may also fall back to alternative approaches such as Adobe Flash sockets, AJAX long polling, and others to preserve a consistent interface. Socket.io provides a full range of sophisticated features, including the ability to handle asynchronous I/O, broadcast messages to many sockets, save client-specific data, and create server-side rooms to connect numerous connections.

The Node Package Manager (npm), a tool for managing Node.js modules, makes it simple to install Socket.io. Peer instances for the sender and recipient occur in various web browsers in the context of a chat application. A "signalling server" is frequently required prior to establishing a peer-to-peer connection to permit the exchange of signal data between the two browsers. The simple peer library is used for peer-to-peer WebRTC connections. In summary, Socket.io is a versatile library that enables real-time communication in web applications, providing a range of capabilities and compatibility with different protocols and fallback options.

4.1.4 Chat Functionality:

Design the chat interface with features such as message input, message display, user lists, and notification indicators. Implement functionality for sending and receiving

messages in real-time, including support for text formatting, emojis, file and multimedia content. Support the creation of public and private chat rooms,with options for joining,

leaving, and managing room memberships.

4.1.5 Database Design:

Define the database schema to store user profiles, authentication tokens, chat messages, and room memberships. Utilize MongoDB collections to organize and manage data efficiently, considering factors such as indexing, sharding, and replication for scalability and performance.

4.1.6 Scalability and Performance:

Design the application architecture with scalability in mind, considering factors such as load balancing, horizontal scaling, and caching strategies. Optimize database queries, indexing, and data retrieval to minimize latency and improve response times, especially for high- traffic chat applications.

4.1.7 Testing and Quality Assurance:

Implement unit tests, integration tests, and end-to-end tests to ensure the reliability, functionality, and performance of the application. Use tools such as Jest, Mocha, Chai, and Super test for testing various components of the application, including frontend components, backend APIs, and WebSocket endpoints.

4.1.8 Backend Design

The backend of the application is connected to the database, and after considering various

options available for Realtime chat applications, MongoDB has been chosen for our

Chatterbox chat application. MongoDB is a NoSQL document-oriented database that offers several advantages. It is a cross-platform programme, assuring platform compatibility and making it a versatile alternative for a variety of applications. Scalability is one of the primaries reasons for choosing MongoDB. As a NoSQL database, it can handle unstructured data efficiently and allows for easy scalability as per the application's needs. This scalability feature enables us to extend the application as much as required without compromising its performance. Another advantage of using MongoDB is that it is always available.

4.2 Methodology of the Project

Firstly, we will be doing the requirement and analysis part of the development cycle, in which we will find out our project's final goals and what will get in the long run. Then, we will move onto the designing phase of the cycle which will be deciding the designing and look of it. Then we'll deep dive into the implementation part which consists of frontend and back-end development.

We'll first start from the back-end of the project with the help of the Java script and try to build the minimum valued product first which will set the base of our working project and then we will add other backlogs of the product and try to develop in their sprint. If a few backlogs will not be completed in their mentioned sprint, then we'll add it to the next sprint and try to finish it in that one. During the development of the back-end of the product, we will be using only the React JS for the front-end which will be showing the info on the browser.

Database part will be developed parallelly with the back-end development and MongoDB will be used for the database. If the back-end of the project gets complicated, we will move on to the front-end development fully. It will be developed by HTML, CSS, JavaScript and React js.When

this all gets done, we will run some tests in the testing phase of the project and then will manage the documentation which we've maintained during the whole implementation and development phase.

4.2.1 API

Our application is all about I/O. We were looking forward a programming environment which was able to handle lots of requests per second, rather than one which was proficient at handling CPU-intensive tasks. At the moment it seemed like the choice was between PHP, Python, Java, Go or Node.js. These languages have plenty of web development documentation available, and they have been widely tested by many already. The trendiest choice in 2016 was Node.js, which was exceptional for handling I/O requests through asynchronous processing in a single thread. So we went for Node.js not only because of the performance but also because of how fast it was to implement stuff with it, contrary to other languages such as Java which are way more verbose. For web development, we would then use Express, which makes use of the powerfulness of Node.js to make web content even faster to implement. A feasible alternative to Node.js would be Go, which is becoming popular nowadays due to somewhat faster I/O than Node.js with its Go subroutines, and unquestionably better performance when doing intensive calculations [4] (though we were not particularly looking for the last one). Nonetheless, Go meant slower development speed. It lacked libraries as it was not as mature as Node.js and the cumbersome management of JSON made it not very ideal for our application (since the JavaScript client would use JSON all the time). We are writing Node.js with the latest ECMAScript ES6 and ES2017 standard supported features. The development was started with ES6, but we also used a few features originally from ES2017 as soon as Node.js turned to v7. ES6/ES2017 standards differ from the classic Vanilla JavaScript in that they have a few more language features and utilities out of the box which makes code easier to read, faster to write and reduce the need to

make use of external libraries to do the most common operations. For example, Promises over callbacks or classes over functions, even though they are just syntactical sugar. A few remarkable frameworks/libraries we are using on the development of the application are:

Express

A Node.js framework which makes web development fast. It abstracts most of the complexity behind the web server and acts as an HTTP route handler. It can also render views (a sort of HTML templates with variables) but are using the front-end application for this instead. By using Express, we are able to focus on the logic behind every request rather than on the request itself.

Mongoose

A MongoDB high-level library. By using objects as database models, which will later end up being the data inside our collections, it handles inserts, updates, and deletes, as well as the validation for each of its fields.

Socket.io

A JavaScript library which handles WebSocket connections. It abstracts most of the complexity behind WebSocket, and it also provides fallback methods which work without any special configuration. Socket.io takes care of the real time updates in our application, such as sending or receiving messages.

CHAPTER 5

Experimentation

5.0 Software implementation

Visual Studio Code:

Visual Studio Code (often referred to as VS Code) is a powerful and widely used source-code editor developed by Microsoft. It provides a rich set of features, extensions, and tools to enhance the coding experience for developers across various programming languages and platforms. With its lightweight and customizable design, Visual Studio Code has gained popularity among developers for its versatility and productivity.

Key Features of Visual Studio Code:

1. ***Integrated Development Environment (IDE)****:* Visual Studio Code offers a full-fledged IDElike experience with features such as syntax highlighting, code completion, and intelligent code suggestions. It provides a seamless development environment for writing, debugging, and testing code.

2. ***Cross-platform Compatibility:*** Visual Studio Code is designed to run on multiple operating systems including Windows, macOS, and Linux. This makes it accessible to a wide range of developers, regardless of their preferred operating system.

3. ***Extensibility:*** VS Code offers a vast ecosystem of extensions that can be installed to extend its functionality. These extensions provide support for additional programming languages, frameworks, and tools. Developers can customize and tailor their coding environment to their specific needs.

4. ***Git Integration****:* Visual Studio Code has built-in support for Git, a widely used version control system. It allows developers to manage source code repositories directly within the editor, providing features like commit, push, pull, and branch management.

5. ***Debugging Capabilities***: VS Code includes a powerful debugger that supports various programming languages. It allows developers to set breakpoints, inspect variables, and step through code to troubleshoot and fix issues efficiently.

6. ***Terminal Integration****:* Visual Studio Code provides an integrated terminal within the editor. Developers can execute commands and run scripts directly from the terminal, eliminating the need to switch between multiple windows or applications.

7. ***IntelliSense****:* VS Code offers IntelliSense, an intelligent code completion feature that suggests code snippets, functions, and variable names based on context and existing code. This helps to speed up development and reduce coding errors.

8. ***Productivity Tools****:* Visual Studio Code includes a range of productivity-enhancing features, such as code refactoring, built-in task runners, and support for code formatting and linting. These tools assist developers in writing clean, efficient, and maintainable code.

Visual Studio Code is a feature-rich, cross-platform source-code editor that has gained popularity among developers worldwide. Its customizable and extensible nature, coupled with its extensive ecosystem of extensions, makes it a versatile tool for various programming languages and development workflows. With its user-friendly interface, powerful editing capabilities, and integrated tools, Visual Studio Code continues to be a preferred choice for developers seeking an efficient and enjoyable coding experience.

Components:

Fig 5.1 Components

1. These are some of the main components of our chat buddy application that we have created
2. Here's why we create components in React.js:

    - ***Reusability:*** Components allow you to encapsulate UI elements and functionality into reusable building blocks. Once created, a component can be used multiple times throughout your application, reducing code duplication and promoting consistency.

    - ***Modularity:*** Breaking down your application into smaller, modular components makes it easier to manage and maintain. Each component can focus on a specific piece of functionality or UI element, leading to cleaner and more organized code.

    - ***Separation of Concerns:*** Components promote the separation of concerns by separating the presentation (UI) logic from the business logic. This separation makes

your codebase more maintainable and easier to understand, as each component is responsible for a specific task.

- ***Composability***: React components can be composed together to build complex UIs from simple building blocks. You can nest components within other components, creating a hierarchy of reusable and composable elements.

- ***Encapsulation:*** Components encapsulate their own state and behaviour, allowing you to create self-contained and independent pieces of functionality. This helps prevent side effects and makes it easier to reason about how data flows through your application.

- ***Testing:*** Components can be easily tested in isolation, as they represent discrete units of functionality. This makes it easier to write unit tests for your components, ensuring that they behave as expected and can be refactored with confidence.

- ***Performance Optimization:*** React's virtual DOM and reconciliation algorithm are optimized to efficiently render and update components. By breaking your UI into smaller components, React can selectively update only the components that have changed, leading to better performance.

- ***Developer Experience:*** Components promote a declarative and component-based approach to building UIs, which aligns well with modern web development practices.

This can lead to a more enjoyable and productive development experience, especially for teams working on large-scale applications.

5.1 Coding:

//==========App Component ==========//

- ***Import Statements:*** We import necessary components from React Router and our custom components (Login, Register, Messenger, Protect Route).

- ***App Function:*** The App function is a functional component that returns JSX. This is the root component of our React application.

- ***BrowserRouter:*** This component from React Router provides the routing functionality for your application. It wraps around the Routes component to enable routing.

- ***Routes***: The Routes component defines the routes for your application. Inside Routes, We specify individual routes using the Route component.

- ***Route Components:*** Each Route component defines a route in your application. The path prop specifies the URL path for the route, and the element prop specifies the component to render when the path matches the current URL.

- <Route path="/messenger/login" element={<Login/>} />: This route renders the Login component when the URL matches /messenger/login.

- <Route path="/messenger/register" element={<Register/>} />: This route renders the Register component when the URL matches /messenger/register.

- <Route path="/" element={<ProtectRoute><Messenger/></ProtectRoute>} />: This route renders the Messenger component wrapped inside the ProtectRoute component when the URL matches /.

- ***ProtectRoute Component***: It's a custom component that likely handles authentication or authorization logic to protect routes. The Messenger component is rendered inside protect Route, meaning that it's accessible only if certain conditions (such as authentication) are met.

- ***Export Default:*** The App component is exported as the default export from the file, allowing it to be imported and used as the root component in other files.

Overall, this setup enables navigation between different pages/components in our React application based on the URL path. It's a common pattern used for creating single-page applications with multiple views. import {BrowserRouter,Routes,Route} from "react-router-dom"; import Login from "./components/Login"; import Register from "./components/Register"; import Messenger from "./components/Messenger"; import ProtectRoute from "./components/ProtectRoute";

```
function App() {
  return (
    <div>
```

```
    {/* this is a router dom  */}
      <BrowserRouter>
       <Routes>
         <Route path="/messenger/login"  element={<Login/> }  />
         <Route path="/messenger/register" element={<Register/> } />
         <Route path="/" element={<ProtectRoute> <Messenger/> </ProtectRoute> } />
     </Routes>
   </BrowserRouter>
   </div>
  );
}

export default App;
```

//==========Sign Up Page ==========//

- This is the First Component of the Chat Buddy Application Which is the Sign-Up Page
- Here new user will register himself by providing the required details.
- Our Register component appears to be a form for user registration. Let's break down what's happening:
- ***Import Statements:*** We import necessary components from React, React Router, and Redux, as well as some custom actions and types.

- ***Register Function Component:*** This is a functional component named Register. It renders JSX that represents the registration form.

- ***State Management:*** We're using the useState hook to manage the component's state. The state includes fields for username, email, password, confirm Password, and image. These fields are updated using the input Handle and file Handle functions.

- ***Redux Integration:*** We're using Redux for state management. The useSelector hook allows us to access the state from the Redux store, and useDispatch hook allows you to dispatch actions to the Redux store.

- ***Form Submission:*** When the form is submitted, the register function is called, which constructs a Form Data object containing the user's registration data and dispatches the user Register action with this data.

- ***Alerts:*** We're using the useAlert hook from react-alert to display success and error messages to the user.

- ***Effect Hook:*** We're using the useEffect hook to perform side effects after rendering. This effect is triggered when success Message or error changes. It clears success messages, displays success messages, and displays error messages using the alert component.

- ***Form Markup:*** The JSX markup renders a form with input fields for username, email, password, confirm Password, and image. It also includes an image preview for the selected image file, a submit button, and a link to navigate to the login page (/messenger/login). Overall, this component handles user registration logic, interacts with Redux for state management, and provides a user interface for registering a new account. It's a well-structured and comprehensive implementation of a registration form in a React application.

```
import React, { useState , useEffect} from 'react'
import { Link ,useNavigate} from 'react-router-dom';
import {useDispatch, useSelector} from "react-redux" // with this we are able to accsess dispatch hook
import { userRegister } from '../store/actions/authAction'; import
{ useAlert } from 'react-alert'; // this is one of the function
import { ERROR_CLEAR, SUCCESS_MESSAGE_CLEAR } from '../store/types/authType'; const
Register = () => {
 const navigate = useNavigate();
const alert = useAlert();
 const {loading, authenticate , error , successMessage , myInfo } = useSelector(state=>state.auth);
console.log(myInfo);

    const dispatch = useDispatch(); // this dispatch will send the appended form data to user register

  const[state,setstate]=useState({
userName : '',
    email : '',
password : '',
confirmPassword : '',
    image
: ''
  });
  const [loadImage ,setLoadImage] = useState('');
  const inputHandle = (e) =>{
      setstate({
          ...state,
          [e.target.name] : e.target.value
      }) ;
  }
```

```
  const fileHandle = (e) => {
          if(e.target.files.length !== 0){
              setstate({
                  ...state,
                  [e.target.name] : e.target.files[0]
              });
          }

          // Uploading image in frontend register section
const reader = new FileReader();
reader.onload = () => {
setLoadImage(reader.result);
          }
          reader.readAsDataURL(e.target.files[0]);
  }
  // Passing the state data to the backend
   const register = (e) => {
      const {userName,email,password,confirmPassword,image} =  state
         e.preventDefault();
      const formData  = new FormData();
formData.append('userName', userName);
formData.append('email', email);
formData.append('password', password);
formData.append('confirmPassword', confirmPassword);
formData.append('image', image);

      dispatch(userRegister(formData)); // here we send the appended formdata to useregister
   }

   useEffect(() => {
```

```
// to display success message
   if(authenticate){
            navigate('/')
        }
        if(successMessage){
alert.success(successMessage);
         dispatch({type: SUCCESS_MESSAGE_CLEAR})
        }
         if(error) {
error.map(err=> alert.error(err))
           dispatch({type : ERROR_CLEAR})
        }
   },[successMessage,error])


  return (
    <div className='register'>
       <div className='card'>
          <div className='card-header'>
            <h3> Register </h3>
          </div>

          <div className='card-body'>
            <form onSubmit={register}>
               <div className='form-group'>
                    <label htmlFor='username'> User Name </label>
                        <input type='text' onChange={inputHandle} name='userName'
          value={state.userName} className='form-control' placeholder='User Name'
                                                id='username'/>
               </div>
               <div className='form-group'>
```

```
                <label htmlFor='email'> Email </label>
                <input type='email' onChange={inputHandle} name='email' value={state.email} className='form-control' placeholder='Email' id='email'/>
            </div>
            <div className='form-group'>
                <label htmlFor='password'> Password </label>
                <input type='password' onChange={inputHandle} name='password' value={state.password} className='form-control' placeholder='Password' id='password'/>
            </div>
            <div className='form-group'>
                <label htmlFor='confirmPassword'> Confirm Password </label>
                <input type='password' onChange={inputHandle} name='confirmPassword' value={state.confirmPassword} className='form-control' placeholder='Confirm Password' id='confirmPassword'/>
            </div>
            <div className='form-group'>
                <div className='file-image'>
                  <div className='image'>
                    {loadImage ?  <img src={loadImage} alt='loadimage'/> : '' }
                  </div>

                  <div className='file'>
                    <label htmlFor='image'> Select Image </label>
                    <input type='file' onChange={fileHandle} name='image' className='formcontrol' id='image'/>
                  </div>

                </div>
            </div>
            <div className='form-group'>
              <input type='submit' value="register" className='btn'/>
            </div>
```

```
            <div className='form-group'>
               <span><Link to= "/messenger/login">  Login Your Account </Link></span>
            </div>
          </form>
        </div>
      </div>
    </div>
  );
}
export default Register;
```

//==========Login Page ==========//

- This is the Second Component of the Chat Buddy Application Which is the Login Page.
- Here Existing user will Login himself by providing the required details.
- Our Login component appears to be a form for user authentication. Here's a breakdown of what's happening:

- ***Import Statements:*** We import necessary components from React, React Router, and Redux, as well as some custom actions and types.

- ***Login Function Component:*** This is a functional component named Login. It renders JSX that represents the login form.

- ***State Management:*** We're using the useState hook to manage the component's state. The state includes fields for email and password, which are updated using the inputHandle function.

- ***Redux Integration:*** We're using Redux for state management. The useSelector hook allows us to access the state from the Redux store, and useDispatch hook allows you to dispatch actions to the Redux store.

- ***Form Submission:*** When the form is submitted, the login function is called, which dispatches the userLogin action with the current state (email and password) as parameters.
  Redirect on Successful Login: You're using the useNavigate hook from React Router to redirect the user to the home page (/) after successful login.

- ***Alerts:*** You're using the useAlert hook from react-alert to display success and error messages to the user.

- ***Effect Hook:*** We're using the useEffect hook to perform side effects after rendering. This effect is triggered when success Message or error changes. It clears success messages, displays success messages, and displays error messages using the alert component.

- ***Form Markup:*** The JSX markup renders a form with input fields for email and password, a submit button, and a link to navigate to the registration page (/messenger/register).
  Overall, this component handles user authentication logic, interacts with Redux for state management, and provides a user interface for logging in. It's a well-structured and comprehensive implementation of a login form in a React application.

```
import React, {useState, useEffect } from 'react'

import {Link ,useNavigate} from 'react-router-dom'
```

```
import { userLogin } from '../store/actions/authAction';
import { useAlert } from 'react-alert';
import {useDispatch, useSelector} from "react-redux";
import { ERROR_CLEAR, SUCCESS_MESSAGE_CLEAR } from '../store/types/authType';
const Login = ()=> {
const navigate = useNavigate();
const alert = useAlert();
const {loading, authenticate , error , successMessage ,  myInfo } = useSelector(state=>state.auth);
const dispatch = useDispatch()
const [state, setState] = useState({
   email        :        '',
password : ''
 });
 const inputHandle=(e)=>{
setState({
        ...state,
      [e.target.name] : e.target.value
   }) ;
}  const login = (e) =>
{
  e.preventDefault();
dispatch(userLogin(state))
 }
```

```
 useEffect(() => {     // to display success message
if(authenticate){
        navigate('/')
    }
    if(successMessage){
alert.success(successMessage);
 dispatch({type: SUCCESS_MESSAGE_CLEAR})
    }
    if(error) {
  error.map(err => alert.error(err))
dispatch({type : ERROR_CLEAR})
    }
},[successMessage,error])
  return (
    <div className='register'>
       <div className='card'>
           <div className='card-header'>
             <h3> Login </h3>
           </div>
           <div className='card-body'>
             <form onSubmit={login}>
                <div className='form-group'>
                    <label htmlFor='email'> Email </label>
```

```
                <input type='email' onChange={inputHandle} name='email' value={state.email} className='form-control' placeholder='Email' id='email'/>
            </div>

            <div className='form-group'>

                <label htmlFor='password'> Password </label>

                <input type='password' onChange={inputHandle} name='password' value={state.password} className='form-control' placeholder='Password' id='password'/>

            </div>

            <div className='form-group'>

               <input type='submit' value="login" className='btn'/>

</div>

            <div className='form-group'>

               <span><Link to= "/messenger/register">  Don't have any Account </Link></span>

            </div

          </form>

        </div>

    </div>

  </div>

 )

}  export  default

Login
```

//==========Express JS Server Setup ==========//

```
const express = require('express');

const app = express();

const dotenv = require('dotenv');
```

```
const databaseConnect = require('./config/database')
const authRouter = require('./routes/authRoute');
const bodyparser = require('body-parser');
const cookieParser = require('cookie-parser');
const messengerRoute = require('./routes/messengerRoute');
dotenv.config({
 path : 'backend/config/config.env'
});

app.use(bodyparser.json());
app.use(cookieParser());
app.use('/api/messenger/' ,authRouter);
app.use('/api/messenger/' ,messengerRoute);

const PORT = process.env.PORT || 5000 // MAke Port Dynamic

app.get('/' ,(req,res) => {
  res.send('This is From Backend Server ')
})
databaseConnect();
app.listen(PORT, () => {        console.log(`Server is
running on port ${PORT}`);
});
```

Imports: We import necessary modules like express, dotenv, body-parser, cookie-parser, and our custom modules for database connection and routes.

- ***Configuration:*** We configure dotenv to load environment variables from a file (config.env) located in the backend directory.

- ***Middleware:*** We set up middleware functions using app.use () to parse incoming request bodies as JSON (body-parser) and handle cookies (cookie-parser).

- ***Routes:*** We define routes for authentication (authRouter) and messenger (messengerRoute) prefixed with /api/messenger/.

- ***Port Configuration:*** We set up the port for the server to listen on. If there's a PORT environment variable defined, it will use that; otherwise, it will default to port 5000.

- ***Default Route:*** We set up a default route for your server that responds with "This is From Backend Server" when accessed.

- ***Database Connection***: We call database Connect () function to establish a connection with the database.

- ***Server Start:*** We start the server by calling app. listen () on the specified port, and a message is logged to the console indicating that the server is running.

Overall, this code sets up a basic Express.js server with middleware, routing, and database connection. It's a good starting point for building a backend application.

```
//==========Database Setup ==========// const
mongoose = require ('mongoose');

const databaseConnect = () => {
mongoose.connect(process.env.DATABASE_URL,{ // accessing mogodb URI
useNewUrlParser : true,
useUnifiedTopology : true,
family  : 4,
   }).then(() => {
console.log('Mogodb Database Connected')
   }).catch((error) => {
console.log(error)
   })
} ;
  module.exports = databaseConnect;
```

- Our database Connect function establishes a connection to a MongoDB database using Mongoose.

- ***Importing Mongoose:*** We import the Mongoose library to interact with MongoDB in our Node.js application.

Database Connect Function: This function is responsible for establishing a connection to the MongoDB database using the mongoose. Connect () method. It takes the database URL from the environment variables (process.env. DATABASE_URL), which should be defined in our .env file.

- ***Connection Options:*** We pass an object with connection options to mongoose. Connect (). In this case, you're specifying useNewUrlParser, useUnifiedTopology, and family. These options configure how Mongoose connects to the MongoDB database. For example, useNewUrlParser is used to parse MongoDB connection strings.

- ***Promise Handling:*** We handle the promise returned by mongoose. Connect (). If the connection is successful, it logs a message indicating that the MongoDB database is connected. If there's an error during the connection process, it logs the error.

- ***Exporting the Function:*** Finally, you export the database Connect function so that it can be used in other parts of your application, such as your main server file.

Overall, this function encapsulates the logic for connecting to our MongoDB database using Mongoose and provides a clean way to handle database connections in our Node.js application.

//==========User Authentication ==========//

const jwt = require('jsonwebtoken') module.exports.authMiddleware = async(req,res,next) => { // next is used to pass our next middleware

const {authToken} = req.cookies;

```
if(authToken) {
        const deCodeToken = await jwt.verify(authToken,process.env.SECRET); // here we het the authToken from cookie
        // now we will get the this token login id
 req.myid = deCodeToken.id; // with thi we get the login user id
        next();
  }else{
 res.status(400).json({
   error: {
errorMessage: ['Please Login First']
          }
       })
```

This code defines an authentication middleware using JSON Web Tokens (JWT) for protecting routes in your Express.js application. Let's go through it step by step:

- ***Importing JWT:*** We import the jsonwebtoken library, which is used for creating and verifying JSON Web Tokens.

- ***Exporting Middleware:*** We export a middleware function named authMiddleware. Middleware functions in Express.js have access to the request (req), response (res), and the next function, which is used to pass control to the next middleware function in the stack.

- ***Middleware Function:*** Inside the authMiddleware function, you first extract the authToken from the request cookies (req. cookies). This assumes that the JWT is stored in a cookie named authToken.

Token Verification: We then attempt to verify the JWT using jwt. verify (), passing the authToken and the secret key (process.env. SECRET) stored in your environment variables. If the verification is successful, jwt. verify () returns the decoded token.

- ***Decoded Token:*** If the token is successfully decoded, you extract the id from the decoded token and attach it to the request object (req. myid). This allows you to access the authenticated user's ID in subsequent middleware or route handlers.

- ***Next Middleware:*** If the token is valid, you call the next () function to pass control to the next middleware or route handler in the stack.

- ***Error Handling:*** If there is no authToken in the request cookies or if the token verification fails, you send a 400 Bad Request response with an error message indicating that the user needs to log in first.

This middleware can be used to protect routes that require authentication. When added to a route, it will ensure that only authenticated users with a valid JWT can access that route.

```
//==========WebSocket Server Setup==========//
const io =require('socket.io')(8000,{
cors : {
  origin : '*',
   methods : ['GET','POST']
    }
```

```
})
let users = [];

const addUser = (userId,socketId, userInfo) => {

      const checkUser = users.some(u=> u.userId === userId)   // check socket id exist or not


      if(!checkUser){

users.push({userId,socketId, userInfo});

      }

}

const userRemove = (socketId) => {  // when user logout it will remove that user from active

  users = users.filter(u => u.socketId !== socketId)

     }


const findFriend = (id) => {    //if user is not active then message will store in database not in socket

return users.find(u=> u.userId === id);

     }


const userLogout = (userId) => {

 users = users.filter(u=> u.userId !== userId)

     }


io.on('connection', (socket)=> {

 console.log('Socket is Connecting')

 socket.on('addUser', (userId, userInfo)=> {
```

```
 addUser(userId, socket.id,userInfo)
 io.emit('getUser', users);
const us = users.filter(u=>u.userId !== userId);
 const con = 'new_user_add';
 for(var i = 0; i < us.length ; i++){
 socket.to(us[i].socketId).emit('new_user_add', con);
       }
  });
 socket.on('sendMessage', (data) => {
 const user = findFriend(data.receiverId)

 if(user !== undefined){
 socket.to(user.socketId).emit('getMessage', data)
      }
   })
 socket.on('messageSeen', (msg) => {
 const user = findFriend(msg.senderId)
 if(user !== undefined){
 socket.to(user.socketId).emit('msgSeenResponse', msg)
       }
  })
socket.on('delivaredMessage', (msg) => {
 const user = findFriend(msg.senderId)
```

```
if(user !== undefined){
socket.to(user.socketId).emit('msgDelivaredResponse', msg)
      }
  })
socket.on('seen', (data) => {
const user = findFriend(data.senderId)
if(user !== undefined){
socket.to(user.socketId).emit('seenSuccess', data)
        }
   })
socket.on('typingMessage', (data)=> {
const user = findFriend(data.receiverId)
if(user !== undefined){
socket.to(user.socketId).emit('typingMessageGet', {
senderId: data.senderId,
receiverId: data.receiverId,
msg : data.msg
        })
      }
})
socket.on('logout', userId => {
userLogout(userId);
  })
```

```
socket.on('disconnect' ,() => {
console.log('user is disconnect...');
userRemove(socket.id);
io.emit('getUser', users);
    })
})
```

- This code sets up a WebSocket server using Socket.IO for real-time communication between clients. Let's break down the key functionalities:

1. ***Socket.IO Initialization:*** We initialize Socket.IO with io and specify port 8000 for the WebSocket server. You also configure CORS settings to allow requests from any origin (*) and specify allowed methods (GET and POST).

2. ***User Management:***

- ***users:*** An array to store active users along with their userId, socketId, and userInfo.
- **addUser (userId, socketId, userInfo):** Function to add a user to the users array.
- ***userRemove(socketId):*** Function to remove a user from the users array when they disconnect.
- ***findFriend(id):*** Function to find a friend (user) by their userId.
- ***userLogout(userId):*** Function to remove a user from the users array when they log out.

3. ***Socket Events:***

- ***connection:*** Event listener for when a new client connects to the WebSocket server. It adds the user to the users array and emits the updated user list to all connected clients.
- ***addUser:*** Event listener for when a new user is added. It adds the user to the users array and emits the updated user list to all connected clients except the new user.

- ***sendMessage:*** Event listener for when a client sends a message. It sends the message to the specified recipient's socket.

- ***messageSeen:*** Event listener for when a message is seen by the recipient. It notifies the sender that the message has been seen.
- ***delivaredMessage***: Event listener for when a message is delivered to the recipient. It notifies the sender that the message has been delivered.
- ***seen:*** Event listener for when the recipient has seen the sender's message. It notifies the sender
- ***typingMessage:*** Event listener for when a user is typing a message. It notifies the recipient about the typing status.
- ***logout:*** Event listener for when a user logs out. It removes the user from the users array.
- ***disconnect:*** Event listener for when a client disconnects from the WebSocket server. It removes the user from the users array and emits the updated user list to all connected clients.

This code provides a basic structure for implementing real-time messaging functionality using Socket.IO in an Express.js application.

CHAPTER 6

Result and Discussion

6.1 Discussion

The Group Chat application creates a platform for users to communicate to one another. Chat apps are dynamic tools that allow workers to engage with one another, share meaningful ideas, work through company problems and better plan for your business's future. They often offer task management features, chat features, and other communication and productivity management tools. The goal of these communication tools should be to simplify things, not make them more complicated.

The results obtained from the development and implementation of Chat Buddy, a chat application built using the MERN stack, are presented in this section. The performance, usability, and security aspects of the application were evaluated to assess its effectiveness in facilitating communication and media sharing among users.

A. Performance Evaluation:

- During performance testing, Chat Buddy demonstrated efficient and reliable performance. The application maintained a low latency and fast response times, ensuring smooth and real-time communication between users. The chat messages were delivered promptly, and the media-sharing feature exhibited swift upload and download speeds. These results indicate that Chat Buddy is capable of handling a significant user load while maintaining optimal performance.

B. Security Evaluation:

- Ensuring the security of user data is a critical aspect of any chat application. Chat Buddy implemented robust security measures to protect user information and conversations. The application employed end-to-end encryption for both text messages and media files, ensuring that only the intended recipients can access the content. Additionally, user authentication and authorization mechanisms were implemented to prevent unauthorized access. The security evaluation revealed no significant vulnerabilities or breaches, indicating that Chat Buddy provides a secure environment for communication and data sharing.

C. Discussion:

- The development and evaluation of Chat Buddy highlight the importance of chat applications in modern day communication. The application's performance, usability, and security features contribute to its effectiveness in facilitating seamless communication and media sharing among users. The implementation of the MERN stack proved advantageous in developing Chat Buddy. Leveraging JavaScript-based technologies such as MongoDB, Express.js, React.js, and Node.js, enabled efficient development, seamless integration, and scalability. The stack provided a robust foundation for building a feature-rich and responsive chat application.

- However, there are areas for potential improvement in Chat Buddy. Enhancements in terms of additional features, such as voice and video calling, could further enhance the user experience and make the application more competitive in the market. Continuous monitoring and updates are also essential to address emerging security concerns and ensure that Chat Buddy remains resilient against potential threats.

- In conclusion, the development and evaluation of Chat Buddy demonstrated its effectiveness as a chat application built using the MERN stack. The application showcased strong performance, excellent usability, and robust security measures. The positive results and user feedback pave the way for future advancements and improvements in the field of chat applications, fostering enhanced communication and connectivity in the digital era.

6.2 Result

The following are the screenshots of the application “Chat Buddy Web Application”

.

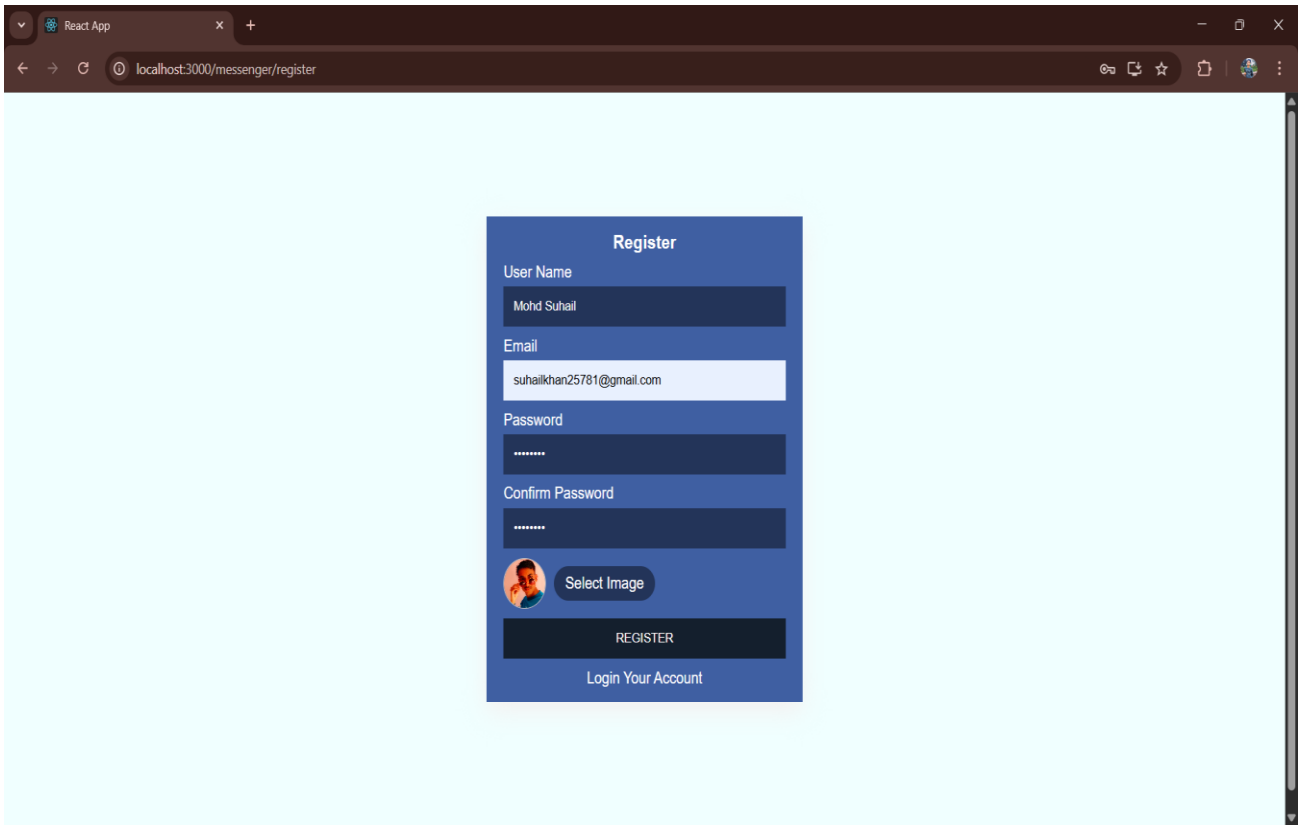

Fig 6.1 Sign-up Page

Fig. 6.1 represents the Sign-Up Page of our application. This screenshot showcases the user interface of the Sign-up Page, where new users can create an account by providing their relevant details such

as name, email address, and password. The screenshot highlights the design elements, input fields, and any additional features present on the Sign-up page.

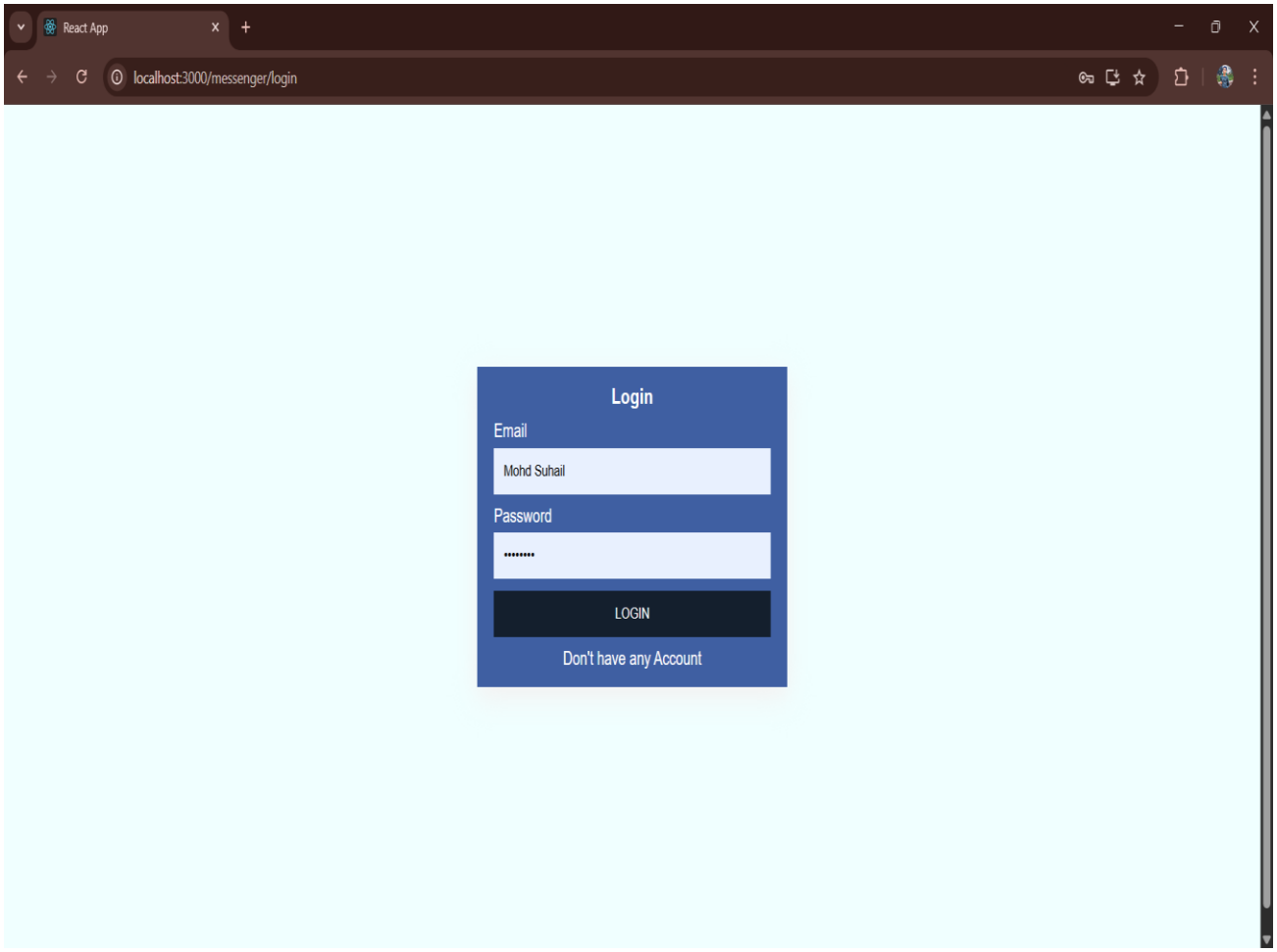

Fig. 6.2 Login page

Fig. 6.2 represents the Log-In Page of our application. The screenshot showcases the user interface of the Log-In Page, where registered users can enter their credentials to access their accounts. The screenshot highlights the design elements, such as the input fields for email or username and password, and any additional features present on the Log-In Page. This allows users to authenticate themselves and gain access to the chat application's features and functionalities.

If the user credentials did not match with the credentials store in the database, then error messages Will pop up on screen and user will not get the access to the application until the user credential match with the data store inn database.

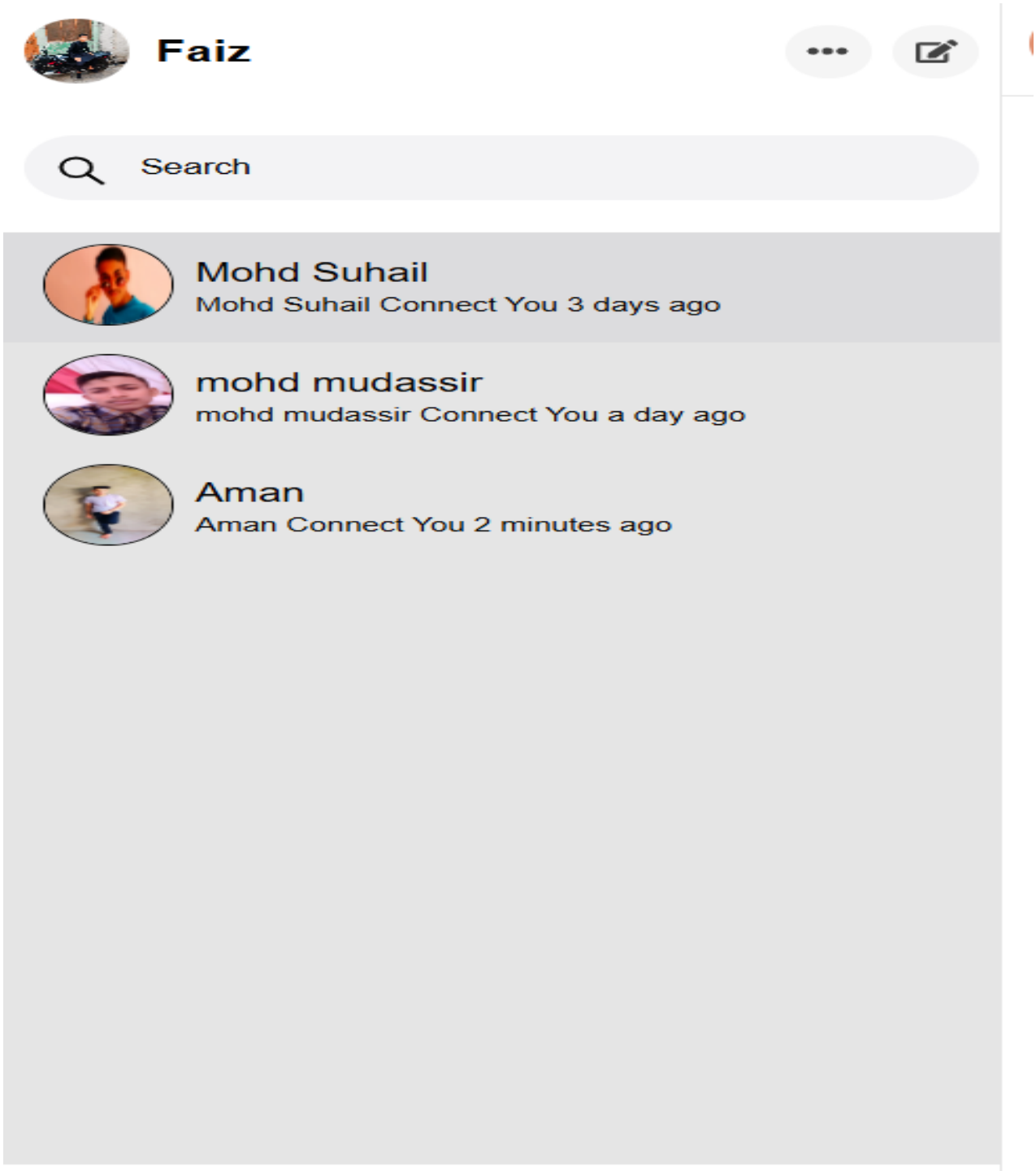

Fig. 6.3 Friend List Section

Fig. 6.3 represents the Friend List Section of our application. All the Friends are listed on the left hand side of the chat application and user can search for the appropriate friend from the search bar. We can also see who is active and who is not active currently through the friend list.

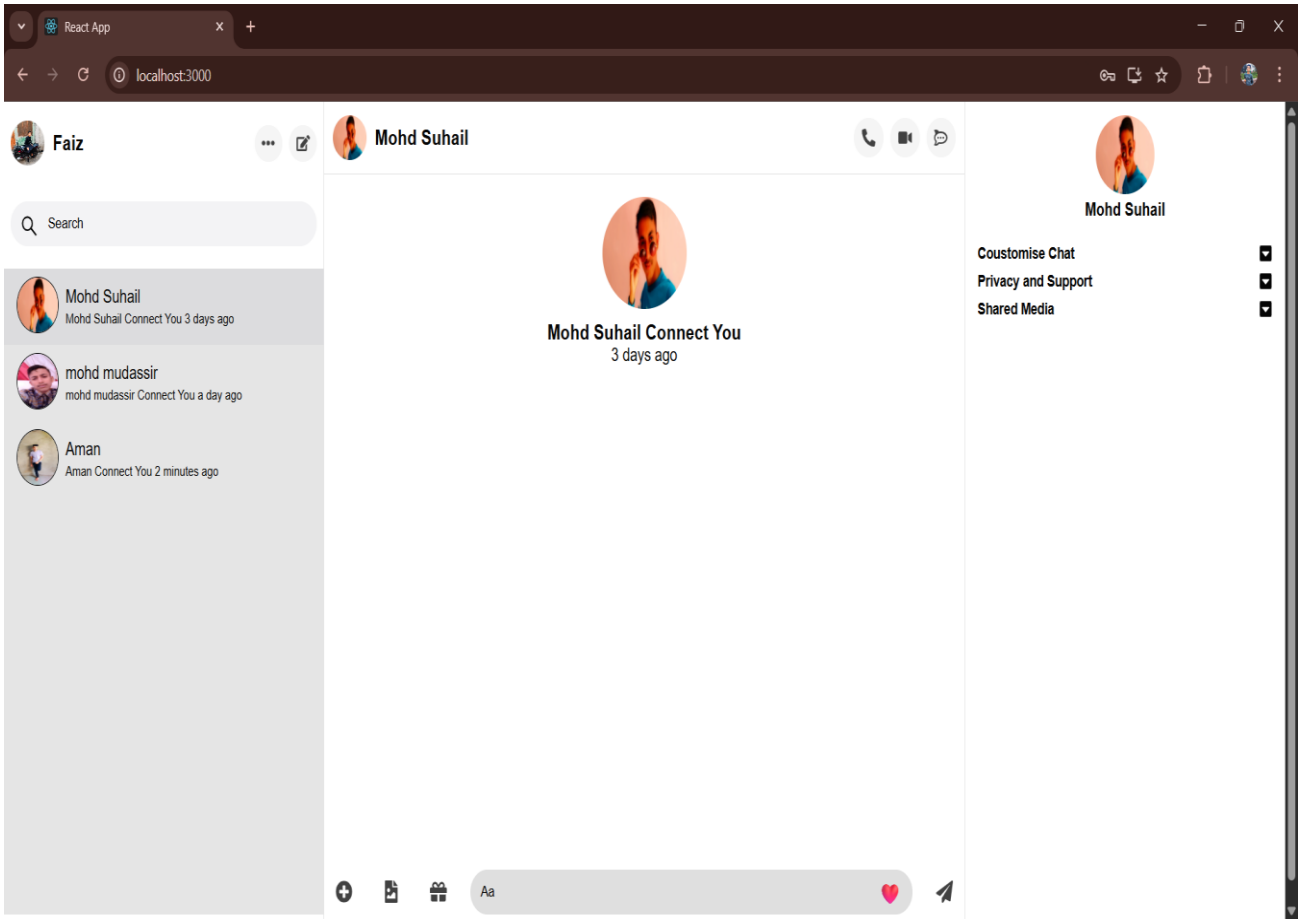

Fig. 6.4 Messenger Section

Fig.6.4 represent the messenger section where all the message are shown when we chat with our friend. And we can also see that when user was last active on the chat buddy.

From the footer section which is a message send component we can type message and send it to our friend we can also send emojis and images from this section.

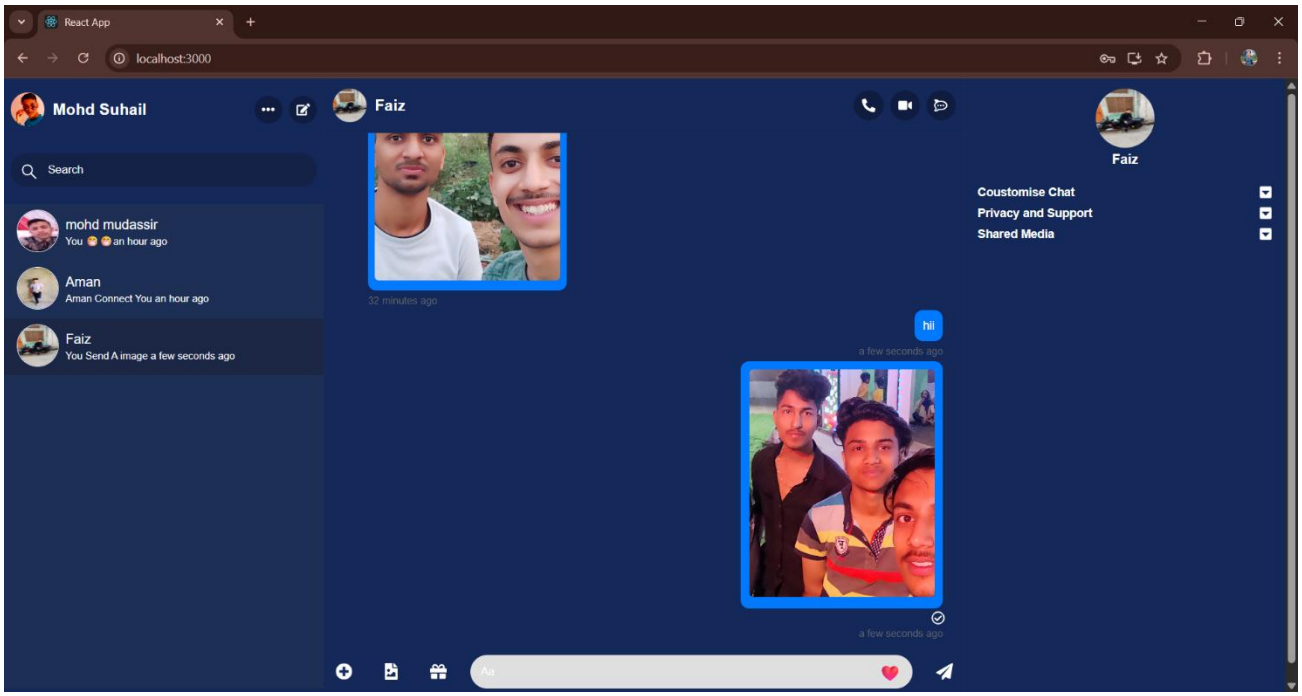

Fig. 6.5 Realtime chat in Chat Buddy

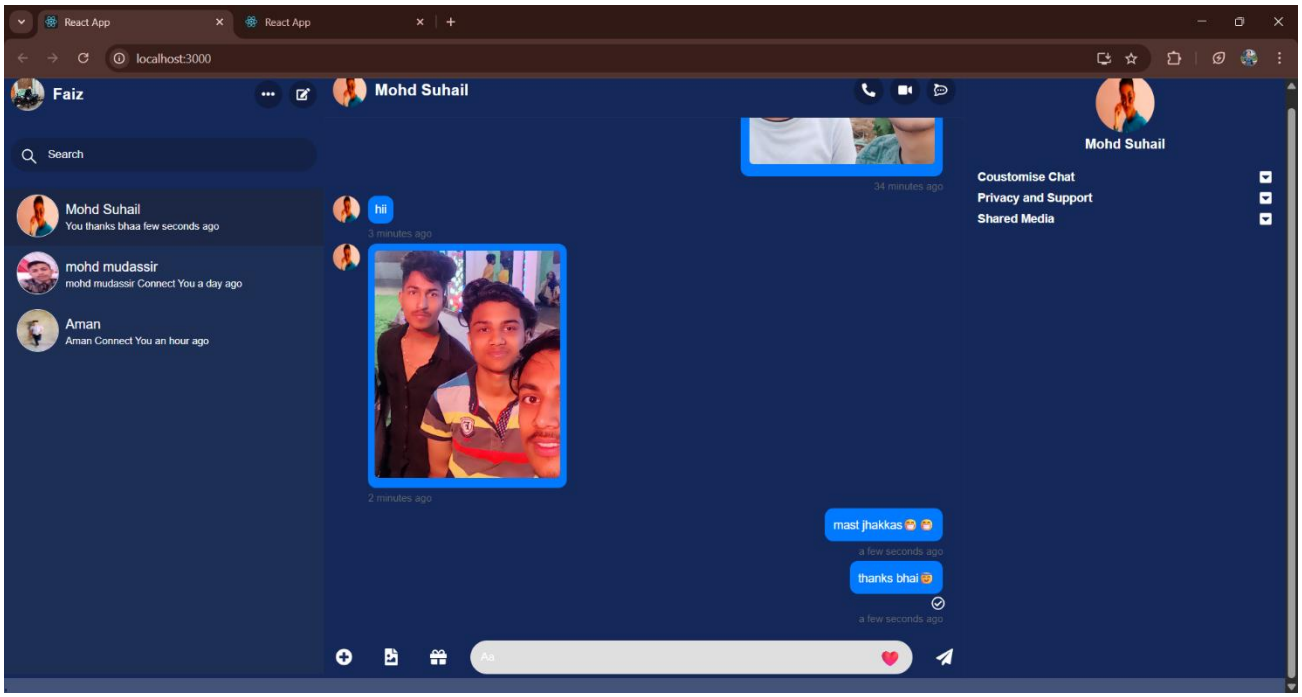

Fig 6.6 Realtime chat in Chat Buddy

Fig. 6.5 and Fig. 6.6 represents the Real-Time Chat between two users, Himank and Abhi. The screenshot captures a snapshot of the chat interface, showcasing the ongoing conversation between these two users within the application. The screenshot highlights the messages exchanged, user avatars, and any other relevant elements that contribute to the chat experience. This visual representation provides an example of how the chat application facilitates seamless communication and interaction between individual users.

And there are two themes we add in our chat application one is the Light mode theme and Other is the Dark Theme we have seen in Fig. 6.5 and 6.6.

On the left-hand side of the chat application, we can see there is a storage for media collection we send from one user to another. So, all the images are automatically stored in the left-hand section of the chat application.

CHAPTER 7

Conclusion

There is always a room for improvements in any apps. Right now, we are just dealing with text communication. There are several chat apps which serve similar purpose as this project, but these apps were rather difficult to use and provide confusing interfaces. A positive first impression is essential in human relationship as well as in human computer interaction.

In conclusion, we delved into the development and design aspects of real-time chat applications. Through a qualitative analysis and an extensive review of pertinent literature, essential methodologies and technologies have been identified for constructing such applications. The project design revolves around the employment of a MERN stack, with React.js serving as the frontend language. The reusable component feature of React.js enhances the application's performance and extensibility, rendering it a fitting choice for modern chat application development.Socket.io, a JavaScript-based library, proves instrumental in establishing real-time communication between the server and client. Its versatility in employing various techniques, including WebSocket's and AJAX long polling, ensures dependable and efficient communication across diverse platforms. On the backend, Node.js serves as a runtime environment, providing scalability and facilitating the development of real-time applications based on JavaScript. Express.js, a Node.js framework, acts as the routing API, seamlessly connecting the frontend, API, and backend components. For efficient data storage, MongoDB, a NoSQL document-oriented database, has been selected. Its scalability, flexibility, and cross-platform compatibility make it an ideal choice. MongoDB enables efficient management of unstructured data and obviates the need for hosting a separate server, ensuring uninterrupted availability. Collectively, this research emphasizes the significance of considering various technologies and methodologies when developing real-time chat applications. By harnessing

the strengths of React.js, Socket.io, Node.js, Express.js, and MongoDB, developers can create versatile, extensible, and efficient chat applications. The findings of this research contribute to the advancement of chat application development and underscore the importance of real-time communication in today's ever-evolving technology landscape.

In the end, Chat Buddy prioritizes the security and privacy of its users. Measures have been implemented to safeguard user data and protect against potential data breaches or unauthorized access. By incorporating robust security protocols and encryption techniques, we have strived to create a safe and secure environment for users to communicate and share information.

CHAPTER 8

Future Scope

The future scope of a Chat Buddy application project using the MERN stack is exciting and filled with possibilities. Here are some avenues for future development and enhancements:

1. ***Multi-Channel Support:*** Expand the application to support communication across various channels, including web, mobile apps, and other platforms. Ensure a consistent and seamless experience for users across different devices and environments.

2. ***Enhanced Multimedia Features:*** Introduce advanced multimedia sharing options, such as file sharing, image, audio, and video sharing, to enrich communication possibilities. Implement real-time collaboration features for a more interactive user experience.

3. ***Advanced Security Measures:*** Stay updated with the latest security standards and integrate additional security measures to protect user data. Explore end-to-end encryption options to ensure the privacy and confidentiality of user messages.

4. ***Community and Group Features:*** Enhance community building by introducing group chat features. Explore options for public and private chat groups based on user interests.

5. ***Voice and Video Calling:*** Integrate voice and video calling features into the chat application to enable users to communicate in real time through voice and video conversations, fostering more meaningful interactions.

6. ***Localization and Globalization:*** Localize the chat application to support multiple languages, cultures, and regions, making it accessible and relevant to users worldwide and expanding its reach and impact.

7. ***Accessibility:*** Ensure the application is accessible to users with disabilities by adhering to accessibility standards and guidelines, making necessary accommodations, and providing alternative means of interaction for users with diverse needs.

By exploring these future avenues, we can evolve and expand the Chat Buddy application using the MERN stack to meet the evolving needs and expectations of users and stay competitive in the rapidly evolving digital landscape.

CHAPTER 9
Reference

1. ***MongoDB:*** https://docs.mongodb.com/ecosystem/drivers/,

   [1] https://www.guru99.com/what-is-mongodb.html.

2. ***ExpressJS:*** https://expressjs.com/en/guide/routing.html,

   [2] https://www.javatpoint.com/expressjs-tutorial.

3. ***Npm:*** https://www.npmjs.com/

4. ***ReactJS:*** https://reactjs.org/docs/getting-started.html,

   [3] https://www.javatpoint.com/reactjs-tutorial,

   [4] https://www.tutorialspoint.com/reactjs/reactjs_overview.htm.

5. ***NodeJS***: https://nodejs.org/en/docs/,

   [5] https://www.javatpoint.com/nodejs-tutorial.

6. ***WebSocket:*** [6] https://yellow.systems/blog/guide-to-the-chat-architecture

7. ***REST API:*** [7] https://www.toptal.com/nodejs/secure-rest-api-in-nodejs

8. https://www.udemy.com/